

Bayesian Neural Network for Reservoir Permeability Prediction

Domain: Oil & Gas — petrophysics / formation evaluation

Reference: [Turing.jl](#) — probabilistic programming in Julia. This notebook follows the structure of Turing's official *Bayesian Neural Network* tutorial, applied to a synthetic well-log dataset.

Goal. Predict reservoir permeability K (mD) from standard well-log curves — porosity (ϕ), gamma ray (GR), deep resistivity (R_d) and water saturation (S_w) — using a small multilayer perceptron whose weights are given priors and inferred with MCMC (NUTS) instead of point-estimated by backpropagation. The payoff is a full posterior predictive distribution over permeability, i.e. calibrated uncertainty bands instead of a single number — directly useful for P10 / P50 / P90 reserve estimation and risked decision-making.

A companion notebook ([02_additive_bayesian_updating.jl](#)) reuses this same model class to demonstrate *additive* learning: folding new well data into an existing posterior without re-fitting on the full history.

```
1 md"""
2 # Bayesian Neural Network for Reservoir Permeability Prediction
3
4 ##Domain:## Oil & Gas - petrophysics / formation evaluation
5
6 ##Reference:## [Turing.jl](https://turinglang.org/) - probabilistic programming in
  Julia.
7 This notebook follows the structure of Turing's official «Bayesian Neural
  Network» tutorial, applied to a synthetic well-log dataset.
8
9
10 ##Goal## Predict reservoir permeability ``K`` (mD) from standard well-log
  curves - porosity (``phi``), gamma ray (``GR``), deep resistivity
11 (``Rt``) and water saturation (``Sw``) - using a small multilayer
12 perceptron whose «weights» are given priors and inferred with MCMC (NUTS)
13 instead of point-estimated by backpropagation. The payoff is a full
14 posterior predictive distribution over permeability, i.e. calibrated
15 uncertainty bands instead of a single number - directly useful for P10 /
16 P50 / P90 reserve estimation and risked decision-making.
17
18
19 A companion notebook ('02_additive_bayesian_updating.jl') reuses this same
20 model class to demonstrate «additive» learning: folding new well data into
21 an existing posterior without re-fitting on the full history.
22 """
```

```
1 begin
2   using Turing
3   using MCMCChains
4   using Distributions
5   using Random
6   using LinearAlgebra
7   using StatsPlots
8   using Plots
9   using DataFrames
10  using PlutoUI
11 end
```

Table of Contents

Bayesian Neural Network for Reservoir Permeability Prediction

- Synthetic well-log dataset
- Train / test split and feature standardization
- Bayesian neural network (Turing.jl)
- Posterior sampling (NUTS)
- Posterior predictive & uncertainty quantification
- Partial-dependence view: permeability vs. porosity
- Discussion

1 [TableOfContents\(\)](#)

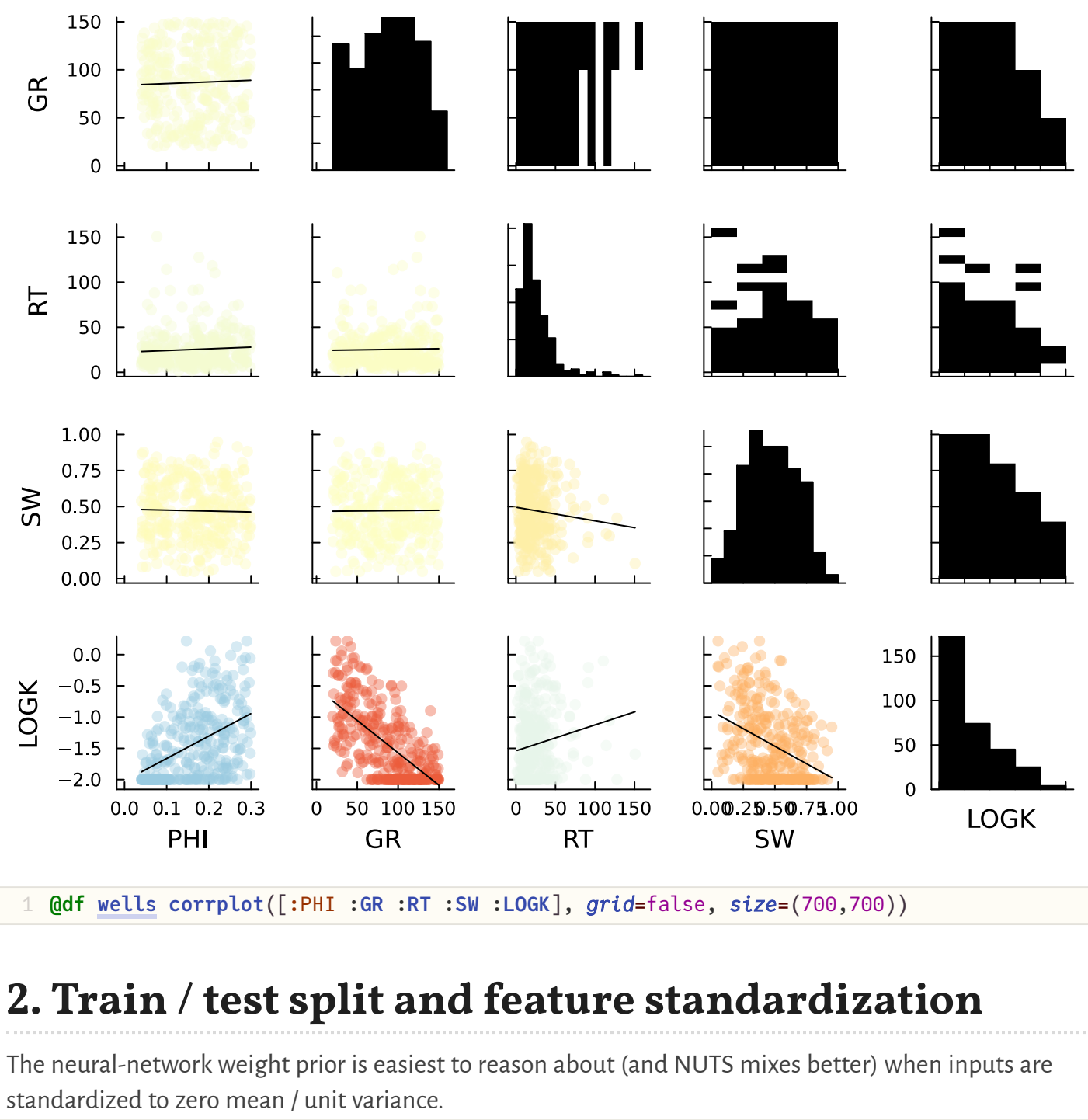
1. Synthetic well-log dataset

Real well-log/core-permeability pairs are commercially sensitive, so we generate a synthetic but petrophysically-motivated dataset: permeability is approximately log-normal and depends *nonlinearly* on porosity and water saturation (a simplified stand-in for Kozeny–Carman-type behaviour), with gamma ray acting as a shale/clay-content proxy and resistivity carrying independent (noisy) information about pore-fluid and texture.

```
1 ## 1. Synthetic well-log dataset
2
3
4 Real well-log/core-permeability pairs are commercially sensitive, so we
5 generate a synthetic but petrophysically-motivated dataset: permeability is
6 approximately log-normal and depends «nonlinearly» on porosity and water
7 saturation (a simplified stand-in for Kozeny–Carman-type behaviour), with
8 gamma ray acting as a shale/clay-content proxy and resistivity carrying
9 independent (noisy) information about pore-fluid and texture.
10 """
11
12 generate_wells (generic function with 1 method)
13
14 function generate_wells(n::Int; seed::Int)
15     rng = MersenneTwister(seed)
16     phi = rand(rng, Uniform(0.04, 0.30), n) # porosity (fraction)
17     gr = rand(rng, Uniform(20.0, 150.0), n) # gamma ray (API)
18     rt = rand(rng, LogNormal(log(20.0), 0.8), n) # deep resistivity (ohm.m)
19     sw = clamp.(rand(rng, Beta(2.0, 2.5), n), 0.05, 0.98) # water saturation
20     (fraction)
21
22     # nonlinear, petrophysically-flavoured log-permeability model
23     log10K = @. (-1.2 + 9.0 * phi - 12.0 * phi^2 - 0.014 * gr -
24         1.6 * sw + 0.9 * sw * phi + 0.35 * log10(rt))
25     noise = rand(rng, Normal(0.0, 0.22), n)
26     log10K = log10K .+ noise
27     K = clamp.(log10K, 0.01, 5000.0) # permeability (mD)
28
29     DataFrame(PHI=phi, GR=gr, RT=rt, SW=sw, K=K, LOGK=log10.(K))
30 end
```

	PHI	GR	RT	SW	K	LOGK
1	0.268087	139.635	18.2372	0.128651	0.126016	-0.899574
2	0.17645	123.17	127.694	0.421147	0.01	-2.0
3	0.243882	129.65	32.1704	0.694662	0.0108651	-1.96397
4	0.0990801	40.8219	12.4552	0.239425	0.234488	-0.629879
5	0.0426035	61.864	14.4984	0.0813751	0.0250451	-1.60128

```
1 begin
2   Random.seed!(20240517)
3   wells = generate_wells(320; seed=20240517)
4   first(wells, 5)
5 end
```



```
1 @df wells corrpLot([:PHI, :GR, :RT, :SW, :LOGK], grid=false, size=(700,700))
```

2. Train / test split and feature standardization

The neural-network weight prior is easiest to reason about (and NUTS mixes better) when inputs are standardized to zero mean / unit variance.

```
1 md"""
2 ## 2. Train / test split and feature standardization
3
4 The neural-network weight prior is easiest to reason about (and NUTS mixes
5 better) when inputs are standardized to zero mean / unit variance.
6 """
```

```
1 (n_train = 256, n_test = 64)
2
3 FEATURES = [:PHI, :GR, :RT, :SW]
4
5 function train_test_split(df::DataFrame; frac_train=0.8, seed=1)
6     rng = MersenneTwister(seed)
7     n = nrow(df)
8     idx = shuffle(rng, 1:n)
9     ntrain = round(Int, frac_train * n)
10    df[idx[1:ntrain], :], df[idx[ntrain+1:end], :]
11 end
12
13 wells_train, wells_test = train_test_split(wells; frac_train=0.8, seed=7)
14 (n_train=nrow(wells_train), n_test=nrow(wells_test))
```

```
1 ((256, 4), (64, 4))
2
3 begin
4     feat_mean = [mean(wells_train[:, f]) for f in FEATURES]
5     feat_std = [std(wells_train[:, f]) for f in FEATURES]
6
7     standardize(df) = hcat([ (df[:, f] .- m) ./ s
8         for (f, m, s) in zip(FEATURES, feat_mean, feat_std) )...]
9
10    Xtrain = standardize(wells_train) # size: (n_train, n_features)
11    Xtest = standardize(wells_test) # size: (n_test, n_features)
12    ytrain = Float64.(wells_train.LOGK) # model the log-permeability
13    ytest = Float64.(wells_test.LOGK)
14 end
```

3. Bayesian neural network (Turing.jl)

A small MLP — 4 inputs → 6 tanh → 6 tanh → 1 linear output — with a $\text{Normal}(0, \sigma_{\text{prior}})$ prior on every weight and bias, flattened into a single parameter vector θ so it can be sampled as one block by NUTS.

```
1 md"""
2 ## 3. Bayesian neural network (Turing.jl)
3
4 A small MLP - 4 inputs -> 6 tanh -> 6 tanh -> 1 linear output - with a
5 'Normal(0, sigma_prior)' prior on every weight and bias, flattened into a
6 single parameter vector 'theta' so it can be sampled as one block by NUTS.
7 """
8
9
10 begin
11     N_IN, N_H1, N_H2 = 4, 6, 6
12     N_W1 = N_IN * N_H1 + N_H1
13     N_W2 = N_H1 * N_H2 + N_H2
14     N_W3 = N_H2 * 1 + 1
15     N_PARAMS = N_W1 + N_W2 + N_W3
16
17     "Unpack a flat parameter vector into the MLP's weight matrices/biases and
18     evaluate it on a (n_features, n_obs) design matrix."
19     function nn_forward(X::AbstractMatrix, theta::AbstractVector)
20         i = 1
21         W1 = reshape(theta[i:N_IN+N_H1-1], N_IN, N_H1); i += N_IN * N_H1
22         b1 = theta[i:N_IN+N_H1-1]; i += N_H1
23         W2 = reshape(theta[i:N_IN+N_H1+N_H2-1], N_H1, N_H2); i += N_H1 * N_H2
24         b2 = theta[i:N_IN+N_H1+N_H2-1]; i += N_H2
25         W3 = reshape(theta[i:N_IN+N_H1+N_H2+1], 1, N_H2); i += N_H2
26         b3 = theta[i]
27
28         h1 = tanh.(W1 * X .+ b1)
29         h2 = tanh.(W2 * h1 .+ b2)
30         out = W3 * h2 .+ b3
31         vec(out)
32     end
33
34     N_PARAMS
35 end
```

```
1 bayes_nn (generic function with 2 methods)
2
3 @model function bayes_nn(X, y; sigma_prior=1.0, obs_sigma=0.30)
4     theta ~ MvNormal(zeros(N_PARAMS), sigma_prior^2 * I(N_PARAMS))
5     preds = nn_forward(X, theta)
6     y ~ MvNormal(preds, obs_sigma^2 * I(length(y)))
7 end
```

4. Posterior sampling (NUTS)

We sample the full weight-space posterior with the No-U-Turn Sampler. This is the step an ordinary (non-Bayesian) neural net replaces with a single backprop-trained point estimate — here we get a distribution over networks.

```
1 md"""
2 ## 4. Posterior sampling (NUTS)
3
4 We sample the full weight-space posterior with the No-U-Turn Sampler. This
5 is the step an ordinary (non-Bayesian) neural net replaces with a single
6 backprop-trained point estimate - here we get a distribution over networks.
7 """
```

	iteration	chain	$\theta[1]$	$\theta[2]$	$\theta[3]$	$\theta[4]$	$\theta[5]$	$\theta[6]$	...	mo
1	151	1	0.368363	-0.416645	-0.711492	0.917014	0.848364	-1.25263		
2	152	1	0.58827	0.119699	-1.06436	1.65272	1.05312	-1.20298		
3	153	1	1.42089	1.38499	-0.297997	0.0630809	0.177831	-1.81395		
4	154	1	1.86716	-0.0504722	-1.0303	0.662535	0.798084	-0.0762169		
5	155	1	0.840894	0.625385	0.7090233	0.509244	1.6785	-1.94658		
6	156	1	0.994123	0.782215	1.59041	0.535535	1.65068	-2.25905		
7	157	1	0.925806	-0.216193	0.191562	0.560778	1.16266	0.65314		
8	158	1	0.988387	0.27833	1.0505	0.384008	0.439265	-0.740582		
9	159	1	0.0838898	0.71102	-0.072703	1.03006	1.1809	0.175155		
10	160	1	-1.18596	0.788134	0.340247	0.0534625	0.536444	1.54974		
									...	more

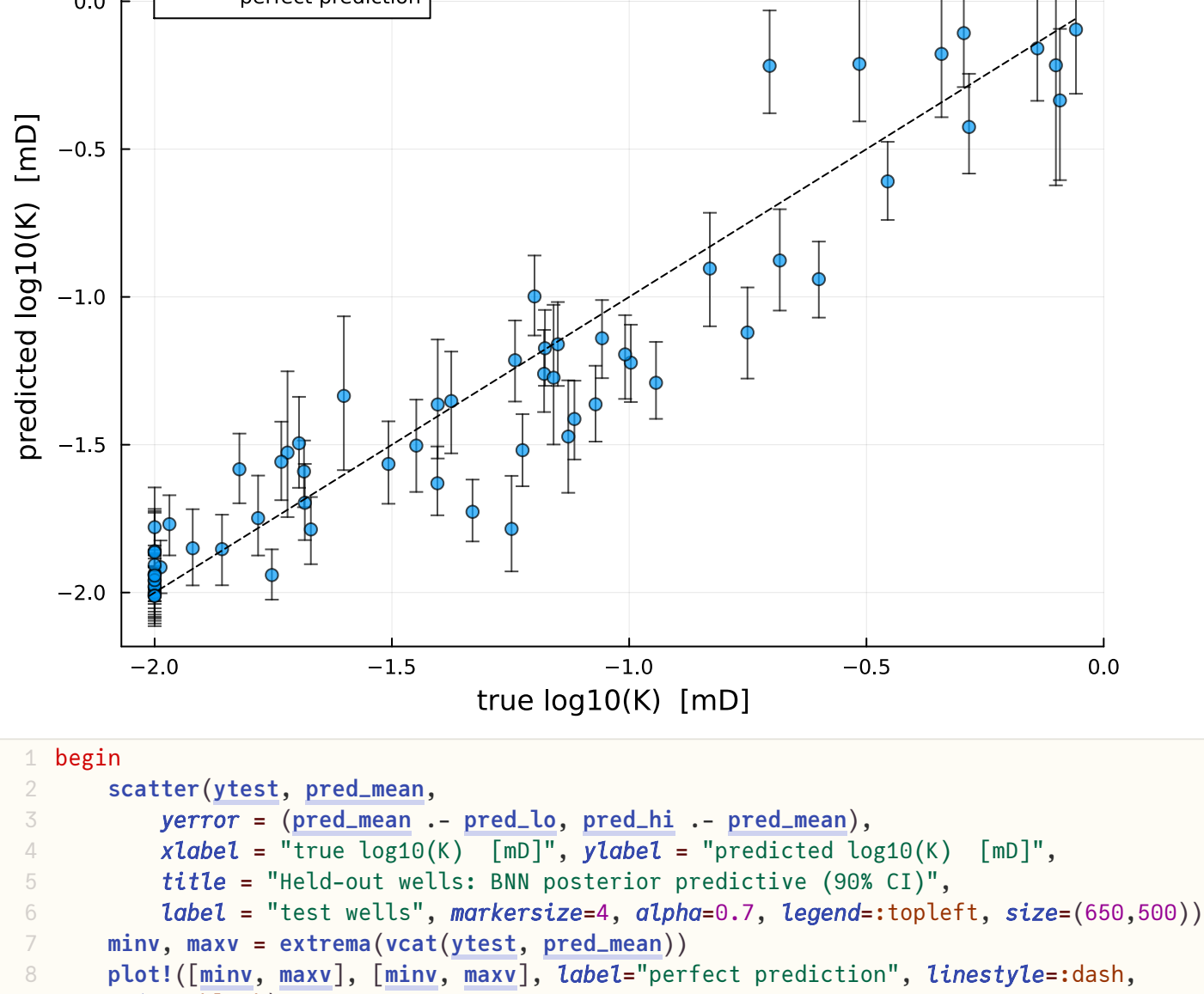
```
1 begin
2   Random.seed!(42)
3   bnn_model = bayes_nn(Xtrain', ytrain)
4   chain = sample(bnn_model, NUTS(0.6), 300; progress=false, chain_type=Chains)
5 end
```

① Found initial size

↳ 0.0125

5. Posterior predictive & uncertainty quantification

```
1 md"""
2 ## 5. Posterior predictive & uncertainty quantification
3 """
4
5
6 (rmse_log10K = 0.194, r2 = 0.895)
7
8 begin
9     theta_samples = Array{group(chain, :theta)} # N_PARAMS x n_samples
10    n_post = size(theta_samples, 2)
11
12    preds_test = reduce(hcat, [nn_forward(Xtest', theta_samples[:, s]) for s in 1:n_post])
13    # preds_test: n_test x n_post
14
15    pred_mean = vec(mean(preds_test, dims=2))
16    pred_lo = [quantile(preds_test[i, :], 0.05) for i in 1:size(preds_test, 1)]
17    pred_hi = [quantile(preds_test[i, :], 0.95) for i in 1:size(preds_test, 1)]
18
19    rmse = sqrt(mean((pred_mean .- ytest).^2))
20    ss_res = sum((ytest .- pred_mean).^2)
21    ss_tot = sum((ytest .- mean(ytest)).^2)
22    r2 = 1 - ss_res / ss_tot
23
24    (rmse_log10K=round(rmse, digits=3), r2=round(r2, digits=3))
25 end
```



```
1 begin
2   scatter(ytest, pred_mean,
3           yerror=(pred_mean .- pred_lo, pred_hi .- pred_mean),
4           xlabel="true log10(K) [mD]", ylabel="predicted log10(K) [mD]",
5           title="Held-out wells: BNN posterior predictive (90% CI)",
6           label="test wells", markersize=4, alpha=0.7, legend=:topleft, size=(650,500))
7   minv, maxv = extrema(vcat(ytest, pred_mean))
8   plot([minv, maxv], [minv, maxv], label="perfect prediction", linestyle=:dash,
9        color=:black)
10 end
```

Partial-dependence view: permeability vs. porosity

Holding gamma ray, resistivity and water saturation at their (training) median, we sweep porosity across its observed range and propagate the full posterior through the network. The widening/narrowing shaded band is the network's own epistemic uncertainty — it is not hand-tuned, it falls out of the posterior.

```
1 md"""
2 ## Partial-dependence view: permeability vs. porosity
3
4 Holding gamma ray, resistivity and water saturation at their (training)
5 median, we sweep porosity across its observed range and propagate the full
6 posterior through the network. The widening/narrowing shaded band is the
7 network's own epistemic uncertainty - it is not hand-tuned, it falls out of
8 the posterior.
9 """
```



```
1 begin
2   phi_grid = range(minimum(wells.PHI), maximum(wells.PHI); length=60)
3   gr_med, rt_med, sw_med = median(wells.train.GR), median(wells.train.RT),
4   median(wells.train.SW)
5
6   Xgrid = zeros(N_IN, length(phi_grid)) # (n_features, n_grid) - matches
7   for k in 1:length(phi_grid)
8       Xgrid[1, k] = (phi_grid[k] - feat_mean[1]) / feat_std[1]
9       Xgrid[2, k] = (gr_med - feat_mean[2]) / feat_std[2]
10      Xgrid[3, k] = (rt_med - feat_mean[3]) / feat_std[3]
11      Xgrid[4, k] = (sw_med - feat_mean[4]) / feat_std[4]
12   end
13
14   grid_preds = reduce(hcat, [nn_forward(Xgrid, theta_samples[:, s]) for s in 1:n_post])
15   grid_mean = vec(mean(grid_preds, dims=2))
16   grid_lo = [quantile(grid_preds[i, :], 0.05) for i in 1:size(grid_preds, 1)]
17   grid_hi = [quantile(grid_preds[i, :], 0.95) for i in 1:size(grid_preds, 1)]
18
19   plot(phi_grid, grid_mean, ribbon=(grid_mean .- grid_lo, grid_hi .- grid_mean),
20        xlabel="porosity (fraction)", ylabel="predicted log10(K) [mD]",
21        label="posterior mean ± 90% CI", title="Permeability vs porosity (GR, Rt, Sw
22        held at median)",
23        size=(650,450), fillalpha=0.25, linewidth=2)
24 end
```

6. Discussion

- The BNN's posterior predictive band gives well-by-well permeability uncertainty, not just a point value — this maps directly onto reserve risking (P10/P50/P90) and completion-design decisions under geological uncertainty.
- chain (an MCMCChains.Chains object) can be inspected with the usual diagnostics (summarystats, trace/density plots via plot(chain), ess_rhat, etc.) to check mixing before trusting the uncertainty bands.
- The next notebook, [02_additive_bayesian_updating.jl](#), shows how to fold new wells into this posterior as they are drilled, without re-running MCMC over the full well history each time.

```
1 md"""
2 ## 6. Discussion
3
4 * The BNN's posterior predictive band gives well-by-well permeability
5 uncertainty, not just a point value - this maps directly onto reserve
6 risking (P10/P50/P90) and completion-design decisions under geological
7 uncertainty.
8
9 * 'chain' (an 'MCMCChains.Chains' object) can be inspected with the usual
10 diagnostics ('summarystats', trace/density plots via 'plot(chain)',
11 'ess_rhat', etc.) to check mixing before trusting the uncertainty bands.
12
13 * The next notebook, '02_additive_bayesian_updating.jl', shows how to fold
14 «new» wells into this posterior as they are drilled, without re-running
15 MCMC over the full well history each time.
16 """
```